Apartment maintenance and billing portal

A Project report submitted for the partial fulfilment of the requirements

to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

K. Amrutha Naga Sri AP24110011514

A. Sirisha AP24110011535

K. Lakshmi Sahitya AP24110011523

P. Vaishnavi AP24110011484

M. Charitha AP24110011597

Instructor

Suriya Saravaran

Department of Computer Science and Engineering

SRM University , AP

May 2026

INTRODUCTION:

The Apartment Maintenance and Billing Portal is a comprehensive full-stack web application designed to streamline and modernize the management of residential apartment communities. Traditional housing societies often rely on manual processes for tasks such as collecting maintenance fees, generating bills, addressing complaints, and managing communication — a approach that frequently results in errors, inefficiencies, and a lack of transparency. This project aims to eliminate these drawbacks by offering a unified digital platform for all such operations.

Developed using the MERN Stack — MongoDB for robust database management, Express.js and Node.js for powerful backend processing, and React.js for a dynamic and responsive front-end experience — the portal empowers administrators to handle a wide range of tasks from a single interface. These include managing flat records, generating and tracking maintenance bills, overseeing payment statuses, resolving resident complaints, and publishing important notices.

Security and reliability are at the core of this system. The portal features strong authentication mechanisms that ensure data protection and session consistency, allowing users to log in and out seamlessly without compromising data integrity. The shift from manual to digital operations not only reduces administrative burden but also promotes greater accuracy and fosters better communication between apartment management and residents.

In essence, the Apartment Maintenance and Billing Portal represents a smart, scalable, and practical solution tailored for the needs of modern residential communities. It stands as a testament to how MERN stack technologies can be effectively leveraged to address and solve everyday real-world challenges.

SCENARIO Based Case Study:

Meet Ravi, an apartment administrator who manages a large residential complex. With hundreds of residents, Ravi often struggles to handle maintenance records, billing calculations, complaint tracking, and communication manually. The traditional system leads to errors, delays, and lack of transparency among residents.

Ravi discovers the Apartment Maintenance & Billing Portal, a full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system is designed to simplify apartment management by automating daily administrative tasks and improving efficiency.

User Authentication and Security: Ravi registers into the system using secure login credentials. The portal ensures safe authentication and allows him to log out and log back in without losing data, maintaining proper session management and data security.

Flat Management: Ravi begins by adding all apartment flat details into the system, including flat numbers, blocks, and floor information. This helps him digitally organize all residential data in a structured manner.

Billing Management: Each month, Ravi generates maintenance bills through the portal. The system allows him to include various charges such as maintenance fees, water bills, electricity charges, parking fees, and any additional costs. This automated billing process reduces manual calculation errors and saves time.

Payment Tracking: The portal maintains a complete record of all payments made by residents. Ravi can easily view transaction history, track pending dues, and ensure transparency in financial management.

Complaint Management: Residents submit complaints regarding maintenance issues such as water leakage, electrical problems, or cleanliness. Ravi can view, manage, and update the status of these complaints, ensuring timely resolution and better service.

Notice Board Communication: Ravi uses the notice board feature to communicate important updates to residents, such as maintenance schedules, events, or emergency announcements. This improves communication and keeps everyone informed.

Feedback System: Residents provide feedback on services through the portal. Ravi analyzes this feedback to improve the quality of maintenance and management services.

Dashboard Overview: The system provides Ravi with a dashboard displaying key information such as total residents, unpaid bills, revenue, and complaints. This helps him make informed decisions quickly.

Ravi's Experience: With the Apartment Maintenance & Billing Portal, Ravi is able to manage the entire apartment system efficiently. The portal reduces manual work, improves accuracy, enhances communication, and ensures transparency. It becomes an essential tool in his daily operations, helping him maintain the apartment smoothly and professionally.

SYSTEM REQUIREMENTS:

1.Software Requirements

These are the essential tools, frameworks, and platforms required to build and run the application effectively.

Operating System: Windows 10/11, macOS, or Linux - Supports cross-platform development and execution.

Node.js (v16 or above): Provides the runtime environment for backend development and server-side logic.

npm (v8 or above): Package manager used to install dependencies for both frontend and backend.

React.js: Used to develop a dynamic, responsive, and component-based user interface.

Express.js: Lightweight backend framework used to build RESTful APIs and handle routing.

MongoDB: NoSQL database used to store apartment data such as residents, flats, bills, payments, complaints, and feedback.

Browser: Google Chrome / Mozilla Firefox (latest versions) .Used for testing and running the frontend application.

Postman: Used to test API endpoints during development.

Visual Studio Code: Preferred IDE with built-in terminal and extensions for MERN stack development.

2. Hardware Requirements

These specifications define the minimum and recommended hardware needed for smooth development and execution.

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or higher - Ensures efficient multitasking and faster build processes.

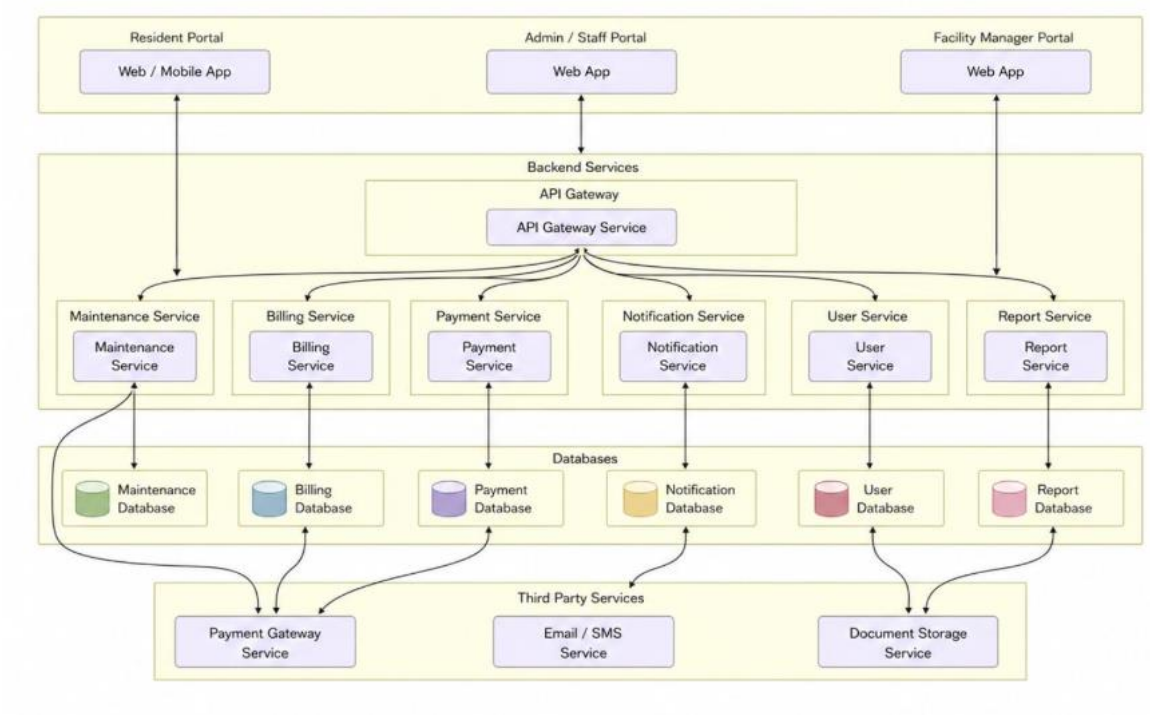
RAM: Minimum 8 GB (16 GB recommended) - Required to run development servers, database, and browser simultaneously.

Storage: At least 2 GB free disk space - Needed for project files, dependencies, and database storage.

Display: Resolution of 1366 × 768 or higher - Recommended for better UI design and coding experience.

PROJECT ARCHITECTURE:

Technical Architecture



In this architecture, the flow begins with a user (represented by the "Admin/User" node) interacting with the Apartment Maintenance & Billing Portal Frontend (represented by the "Frontend (React.js)" node). The user logs into the system and performs actions such as adding flats, generating bills, viewing payments, posting notices, or raising/viewing complaints through the frontend interface.

The frontend then sends HTTP requests (via REST APIs) to the Backend Server (represented by the "Backend (Node.js + Express.js)" node). These requests include operations like authentication, CRUD actions for flats, bill generation, payment updates, complaint submissions, and feedback handling.

The backend processes these requests by applying business logic. It validates the incoming data, authenticates users using JWT/session-based mechanisms, and ensures secure access to the system. Based on the request, the backend interacts with the database layer.

The backend communicates with the Database (represented by the "MongoDB Database" node) to store and retrieve data. This includes:

- User/Admin credentials
- Flat details (block, floor, flat number)
- Billing records (maintenance, water, electricity, etc.)
- Payment history
- Complaints and their statuses
- Notices and feedback

All data is stored in structured collections using Mongoose schemas.

Additionally, the backend may interact with a File Storage System (represented by the "File Storage" node) if the application supports attachments. This is used to store files such as complaint images, payment receipts, or supporting documents. These files can be stored locally or using cloud storage services.

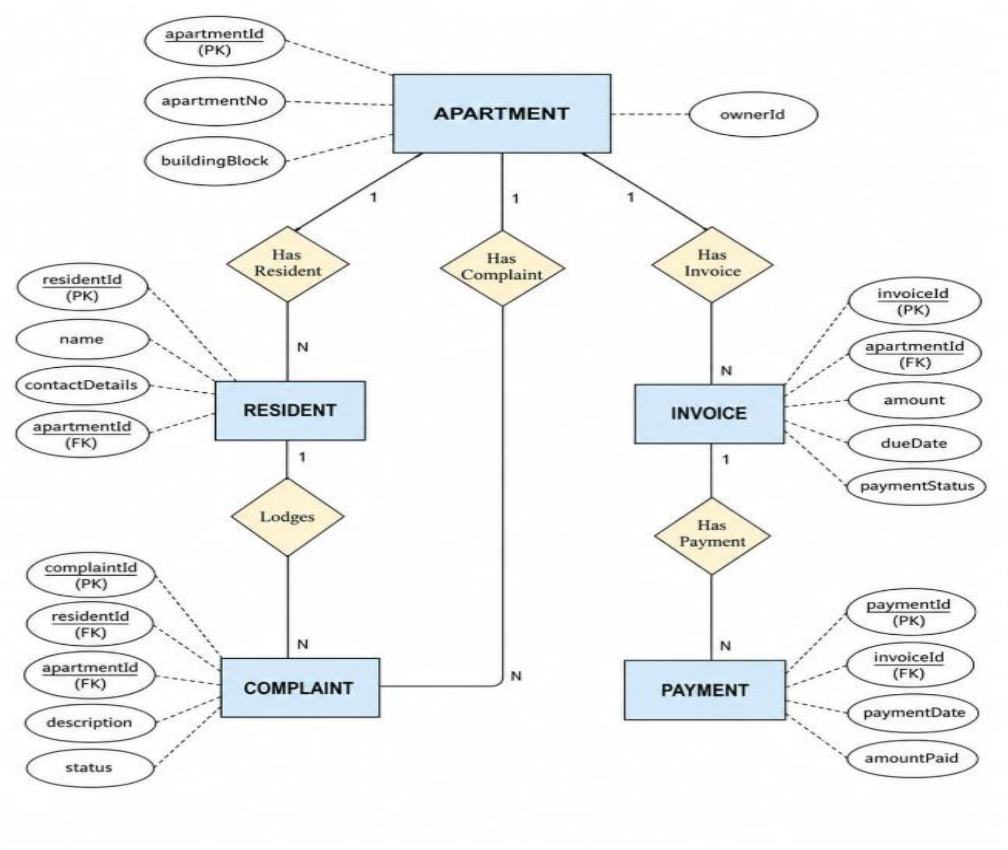
After processing the request, the backend sends a response back to the frontend. This response may include success messages, requested data, or error details. The frontend then updates the UI dynamically and displays the relevant information to the user, such as updated dashboards, payment records, or complaint statuses.

The system follows a client-server architecture with a clear separation of concerns:

- Frontend handles UI/UX
- Backend handles logic and APIs
- Database handles persistent storage

The "style" statements at the bottom are optional and can be used to customize the appearance of specific nodes. In this case, the "Admin/User" and "File Storage" nodes can be styled with a light gray background to visually distinguish external interaction and auxiliary storage components.

ER DIAGRAM



The Entity-Relationship (ER) diagram for the Apartment Maintenance and Billing Portal represents the key entities and their relationships within the apartment management system.

1. Entities:

- Apartment: Represents each apartment unit in the building. Attributes include apartmentId (PK), apartmentNo, buildingBlock, and ownerId.
- Resident: Represents individuals living in the apartment. Attributes include residentId (PK), name, contactDetails, and apartmentId (FK).
- Complaint: Represents issues or complaints raised by residents. Attributes include complaintId (PK), residentId (FK), apartmentId (FK), description, and status.
- Invoice: Represents billing details generated for each apartment. Attributes include invoiceId (PK), apartmentId (FK), amount, dueDate, and paymentStatus.
- Payment: Represents payments made against invoices. Attributes include paymentId (PK), invoiceId (FK), paymentDate, and amountPaid.

2. Relationships:

- Apartment – Resident (Has Resident): One apartment can have multiple residents (1:N relationship). Each resident belongs to one apartment.
- Resident – Complaint (Lodges): One resident can lodge multiple complaints (1:N relationship). Each complaint is raised by a single resident.
- Apartment – Complaint (Has Complaint): One apartment can have multiple complaints (1:N relationship). Each complaint is associated with one apartment.
- Apartment – Invoice (Has Invoice): One apartment can have multiple invoices (1:N relationship). Each invoice belongs to a specific apartment.
- Invoice – Payment (Has Payment): One invoice can have multiple payments (1:N relationship). Each payment is linked to a single invoice.

3. Attributes:

Each entity contains attributes that define its properties.

Example:

Apartment → apartmentNo, buildingBlock

Resident → name, contactDetails

Complaint → description, status

Invoice → amount, dueDate, paymentStatus

Payment → paymentDate, amountPaid

4. Primary Keys:

Each entity has a unique identifier:

Apartment → apartmentId

Resident → residentId

Complaint → complaintId

Invoice → invoiceId

Payment → paymentId

These keys ensure that each record in the system is uniquely identifiable.

5. Foreign Keys:

Foreign keys establish relationships between entities:

Resident.apartmentId → links Resident to Apartment

Complaint.residentId → links Complaint to Resident

Complaint.apartmentId → links Complaint to Apartment

Invoice.apartmentId → links Invoice to Apartment

Payment.invoiceId → links Payment to Invoice

Key Features

The key features of an Apartment Maintenance and Billing Portal can vary depending on its functionality and the needs of the apartment management. However, here are some common key features that such a system might include:

User Authentication and Profiles: Allow admin (or users, if extended) to create accounts, log in securely, and manage their profile information. This ensures that only authorized users can access the system.

Apartment and Resident Management: Allow the admin to add, update, and manage apartment details and resident information, including flat numbers, block details, and contact information.

Billing Management: Enable the admin to generate and manage maintenance bills for each apartment. This includes adding charges such as maintenance, water, electricity, parking, and other expenses.

Payment Management: Facilitate tracking of payments made by residents. It records payment details such as amount paid, payment date, and payment status (paid or pending).

Invoice Generation: Automatically generate invoices for each billing cycle with details like due date, total amount, and payment status.

Complaint Management: Allow residents (or admin) to raise, track, and resolve complaints. The system keeps records of issues along with their current status.

Notice Board / Announcements: Enable the admin to post important notices, updates, and announcements for residents, improving communication within the apartment.

Dashboard and Reports: Provide a dashboard that shows an overview of key data such as total residents, pending payments, total revenue, and complaint status.

Payment History: Maintain a complete record of all transactions and billing history, allowing easy tracking and reference.

Security and Data Management: Ensure secure storage of data using a database system, protecting sensitive information related to residents, payments, and bills.

ROLES AND RESPONSIBILITIES:

User (Resident):

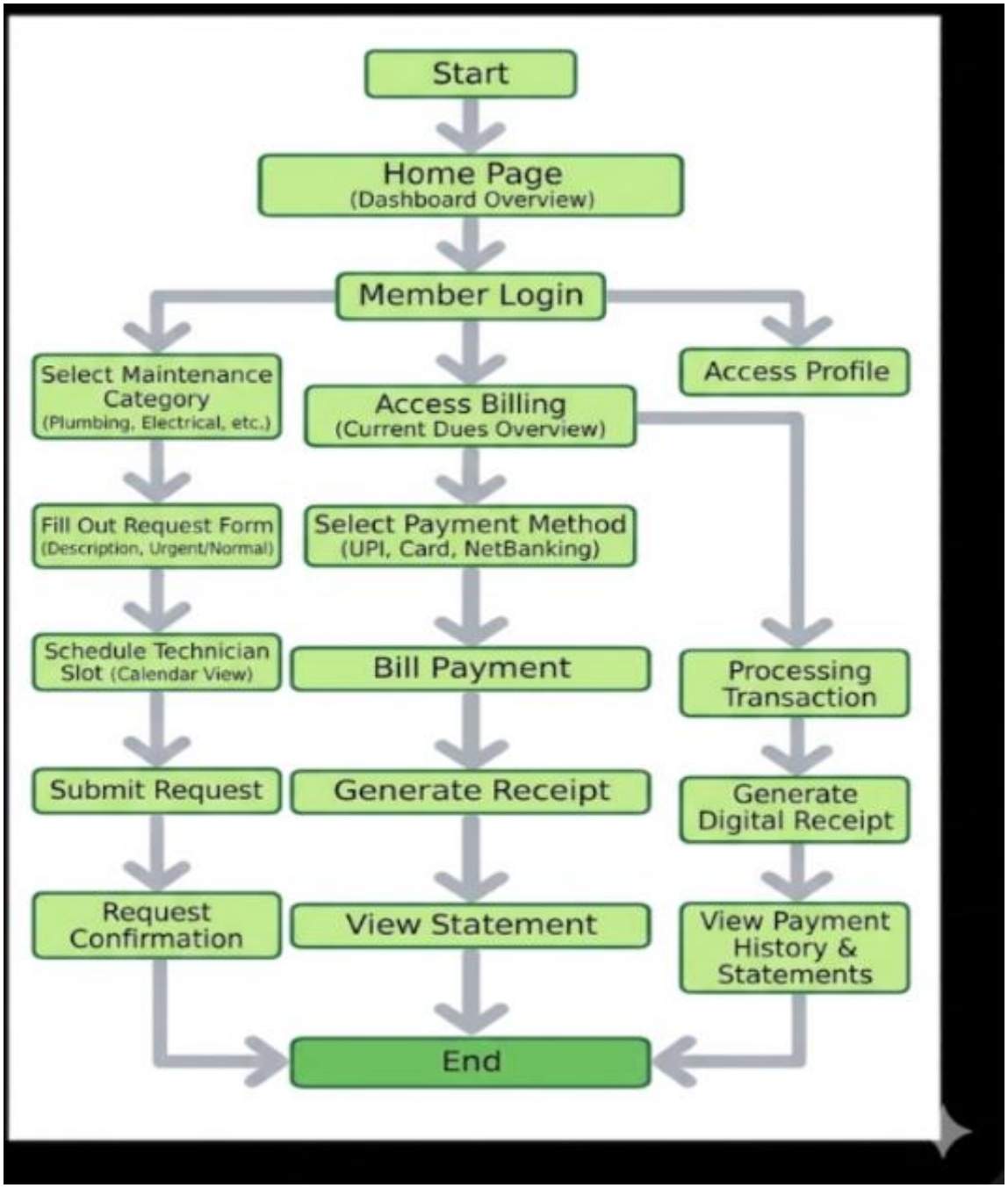
- Registration and Account Management: Residents are responsible for creating an account, providing accurate personal details, and managing their profile, including updating contact information and passwords.
- View Bills and Payments: Residents can view their monthly maintenance bills, including charges for maintenance, water, electricity, parking, and other services. They are responsible for checking due dates and payment status.
- Payment: Residents are responsible for making payments for their generated bills through available payment methods. They must ensure the accuracy of payment details.
- Complaint Management: Residents can raise complaints related to maintenance issues (e.g., water leakage, electricity problems). They can track the status of their complaints until resolution.
- Communication: Residents may communicate with the admin through the system for queries, complaints, or clarifications regarding bills and services.
- Feedback: Residents can provide feedback regarding maintenance services and overall apartment management.

Admin:

- System Management: The admin is responsible for overall system management, including monitoring performance, ensuring data security, and maintaining system configurations.
- Apartment and Resident Management: The admin manages apartment details and resident records, including adding, updating, or removing apartment and resident information.

- Billing Management: The admin generates and manages monthly bills for each apartment, including adding maintenance charges, utilities, and other fees.
- Payment Management: The admin monitors payments, verifies transactions, and updates payment statuses (paid/pending).
- Complaint Management: The admin reviews complaints raised by residents, assigns them for resolution, and updates their status (pending/resolved).
- Notice Management: The admin posts announcements and important notices for residents, such as maintenance schedules or events.

User Flow:



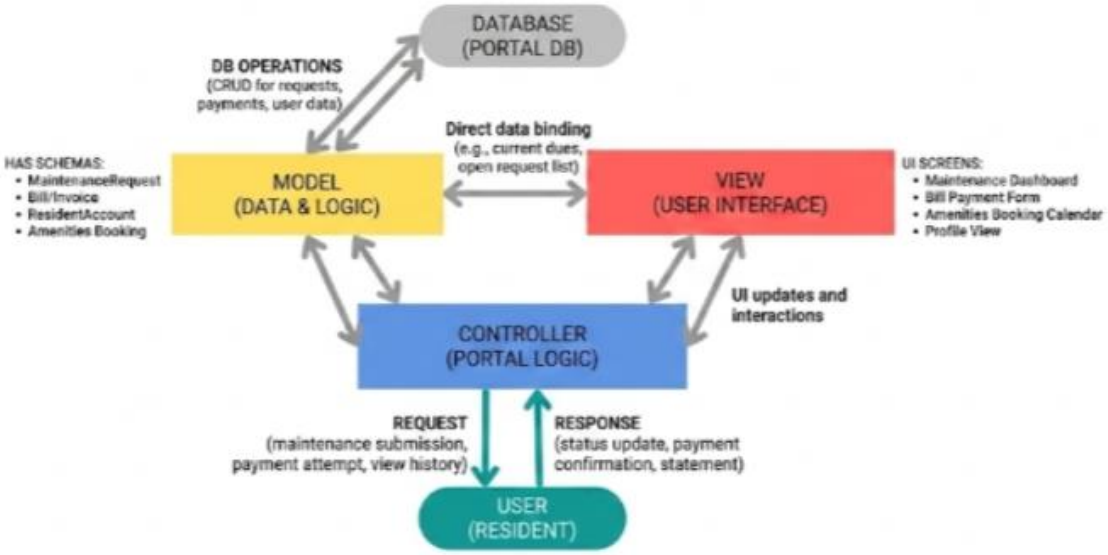
Resident Flow:

- Registration & Account Management – Create and manage account details securely.
- View Bills – Check monthly invoices and dues.
- Payment – Pay maintenance bills using available methods.
- Complaint – Raise and track complaints.
- Communication – Contact admin for queries or issues.
- Feedback – Provide feedback on services.

Admin Flow:

- System Management – Monitor system performance, security, and settings.
- Apartment Management – Add and manage apartment details.
- Resident Management – Manage resident accounts and information.
- Billing Management – Generate and manage invoices.
- Payment Management – Track and verify payments.
- Complaint Management – Handle and resolve complaints.
- Notice Management – Publish announcements and updates.

MVC PATTERN:



The Apartment Maintenance and Billing application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database (CRUD: Create, Read, Update, Delete) for requests, payments, and user data. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Key Schemas:

- MaintenanceRequest
- Bill/Invoice
- ResidentAccount
- Amenities Booking

Controller Layer

The Controller layer acts as an intermediary between the View (routes) and the Model. It receives incoming requests from the resident (e.g., a maintenance submission or payment attempt), processes the input (validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

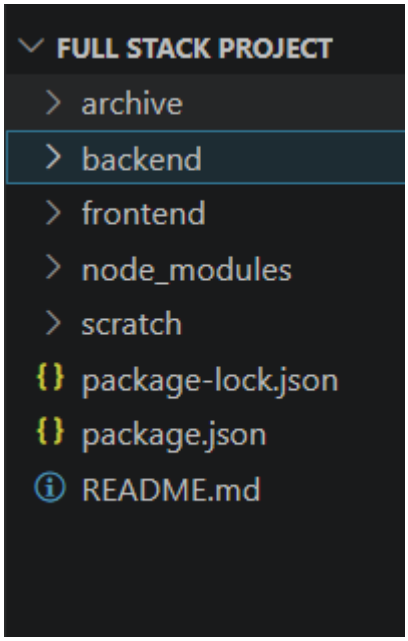
View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.
- Scalability: New features (like a visitor management system) can be added easily by creating new routes, controllers, and models.
- Reusability: Logic in controllers and models can be reused across multiple parts of the application (e.g., the billing logic used by both residents and admins).
- Testing: Each layer can be tested independently, especially the controllers and models.
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers (e.g., frontend vs. backend) without conflict.

Project Setup And Configuration



1. Create a new folder with your project name

(Example: Apartment-Management-System)

2. Inside that folder, create two new folders:

- frontend
- backend

3.Open this main project folder in VS Code

Frontend Setup (Installing React App)

- Open the frontend folder in the terminal of VS Code
- Run the following command to create a React app using Vite:

npm create vite@latest . -- --template react

- Select React framework from the options:

  Select a framework:

  | React

- Select JavaScript variant:

  Select a variant:

  | JavaScript

- Now install all required packages:

  npm install

- To start the React development server:

  npm run dev

Your frontend (UI) will run on:

http://localhost:5173

Backend Setup (Node.js + Express)

- Open the backend folder in terminal of VS Code
- Initialize Node project:

  npm init -y

- Install required dependencies:

  npm install express mongoose cors dotenv

● Create Files in Backend

Create the following file:

server.js

● Create Folders in Backend

Inside backend, create:

models

controllers

routes

● Basic Server Setup (server.js)

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
```

```
app.get("/", (req, res) => {
  res.send("Apartment Management Backend Running...");
});
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

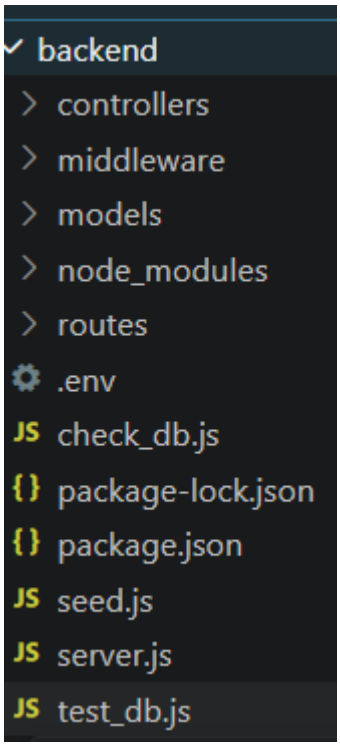
Connecting Frontend & Backend

In your React app (frontend), you can call backend API like:

```
fetch("http://localhost:5000/")
  .then(res => res.text())
  .then(data => console.log(data));
```

BACKEND DEVELOPMENT:

Backend Structure:



controllers

Handles all business logic of the application.

- adminController.js → Handles admin operations such as adding flats, generating bills, managing payments, posting notices, and handling complaints.
- residentController.js → Handles resident-related actions such as viewing bills, making payments, raising complaints, and giving feedback.
- billingController.js → Manages bill generation, updating bill status, and tracking dues.
- paymentController.js → Handles payment processing, payment records, and transaction history.
- complaintController.js → Manages complaint registration, updates, and resolution status.
- noticeController.js → Handles creation and management of notices/announcements.

middleware

Used for handling common functionalities before processing requests.

- authMiddleware.js → Handles authentication and authorization (JWT validation, secure login access).
- errorMiddleware.js → Handles error responses and improves debugging.

models

Defines database schemas using MongoDB (Mongoose).

- Admin.js → Schema for admin login credentials and roles.
- Resident.js → Stores resident details such as name, contact info, and apartment number.
- Apartment.js → Stores apartment/flat details like flat number, block, floor, and owner.
- Bill.js → Stores billing details such as maintenance charges, electricity, water, due date, and payment status.
- Payment.js → Stores payment transactions including amount, date, and payment mode.
- Complaint.js → Stores complaint details such as description, status, and resolution.
- Notice.js → Stores notices/announcements shared by admin.
- Feedback.js → Stores user feedback and ratings.

routes

Defines API endpoints and connects them to controllers.

- adminRoutes.js → Routes for admin actions (login, add flats, generate bills, manage complaints, post notices).
- residentRoutes.js → Routes for resident actions (login, view bills, make payments, raise complaints, give feedback).
- billingRoutes.js → Routes for bill generation and updates.
- paymentRoutes.js → Routes for handling payments and transaction history.
- complaintRoutes.js → Routes for complaint management.
- noticeRoutes.js → Routes for posting and retrieving notices.

Other Important Files

- server.js → Main entry point of the backend; sets up Express server, middleware, and routes.
- .env → Stores environment variables (database URL, secret keys, port number).
- config.js / db.js → Handles database connection setup (MongoDB connection using Mongoose).
- check_db.js / test_db.js → Scripts to verify database connection and test queries.
- seed.js → Used to insert initial/sample data into the database.
- package.json → Contains project dependencies and scripts.
- package-lock.json → Locks dependency versions for consistency.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose.

  In your Node.js project directory, run:

  npm install mongoose

- Create database connection.
- Create Schemas & Models

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

•

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
  } catch (error) {
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. Use the data from the ER diagrams to create the schemas.The Schemas for this application look alike to the one provided below.

1)server.js

```
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');

dotenv.config();

const app = express();
app.use(express.json());
app.use(cors());

// Routes will be imported here
const authRoutes = require('./routes/auth');
const flatRoutes = require('./routes/flats');
const billRoutes = require('./routes/bills');
const paymentRoutes = require('./routes/payments');
const noticeRoutes = require('./routes/notices');
const complaintRoutes = require('./routes/complaints');
```

2)check_db.js

```
const mongoose = require('mongoose');
require('dotenv').config();

mongoose.connect(process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/apartment-management')
  .then(async () => {
    const Bill = require('./models/Bill');
    const bills = await Bill.find();
    console.log("Bills in DB:", bills.length);
    console.log(bills);
    process.exit(0);
  })
  .catch(err => console.log(err));
```

3)seed.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const User = require('./models/User');
const Flat = require('./models/Flat');
const Bill = require('./models/Bill');
const Payment = require('./models/Payment');
const Notice = require('./models/Notice');
const Complaint = require('./models/Complaint');
require('dotenv').config();

const seedDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/apartment-management');
    console.log('Connected to DB for seeding...');

    // Clear existing
    await User.deleteMany();
    await Flat.deleteMany();
    await Bill.deleteMany();
```

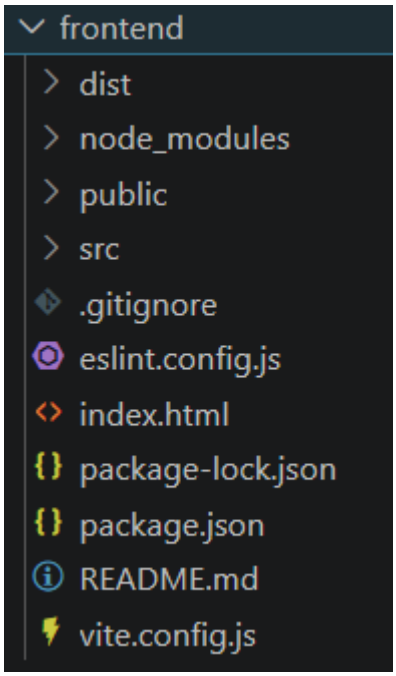
4)test_db.js

```
const mongoose = require('mongoose');
const Bill = require('./models/Bill');
const Payment = require('./models/Payment');
const Notice = require('./models/Notice');

mongoose.connect('mongodb://127.0.0.1:27017/apartment-management')
  .then(async () => {
    console.log("Connected to DB");
    const bills = await Bill.find();
    console.log(`Bills count: ${bills.length}`);
    const payments = await Payment.find();
    console.log(`Payments count: ${payments.length}`);
    const notices = await Notice.find();
    console.log(`Notices count: ${notices.length}`);
    mongoose.connection.close();
  })
  .catch(err => console.log(err));
```

FRONT-END DEVELOPMENT:

Frontend Structure



dist

- Contains the production build files generated after running:

  npm run build

- Optimized HTML, CSS, and JavaScript files for deployment
- Not used during development

node_modules

- Stores all installed dependencies (React, Vite, libraries, etc.)
- Automatically created when running:

  npm install

- No need to modify manually

public

- Contains static assets
- Files inside this folder are directly accessible in the browser

Examples:

- Images (icons, logos)
- Static files like favicon

src

Main working directory of the React application

Inside src:

- App.jsx → Main component that controls overall UI structure
- main.jsx → Entry point of React app (renders App component into DOM)
- components/ → Reusable UI components
- Examples: Navbar, Sidebar, Dashboard cards, Forms
- pages/ → Different pages/screens of the application

Examples:

- Dashboard
- Billing Page
- Payment History
- Complaint Page
- Notice Board

services/ → Handles API calls to backend (fetch/axios requests to backend server)

assets/ → Stores images, icons, styles

styles/ → CSS or styling files

index.html

- Main HTML file loaded in browser
- Contains root where React app is mounted

package.json

- Contains project dependencies and scripts

Examples:

JSON

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

package-lock.json

- Locks versions of installed packages
- Ensures consistency across environments

vite.config.js

- Configuration file for Vite
- Used for:

  Port setup

  Proxy (frontend → backend connection)

  Build settings

eslint.config.js

- Configuration for code quality and linting
- Helps maintain clean and error-free code

.gitignore

- Specifies files/folders to ignore in Git

Example:

node_modules

dist

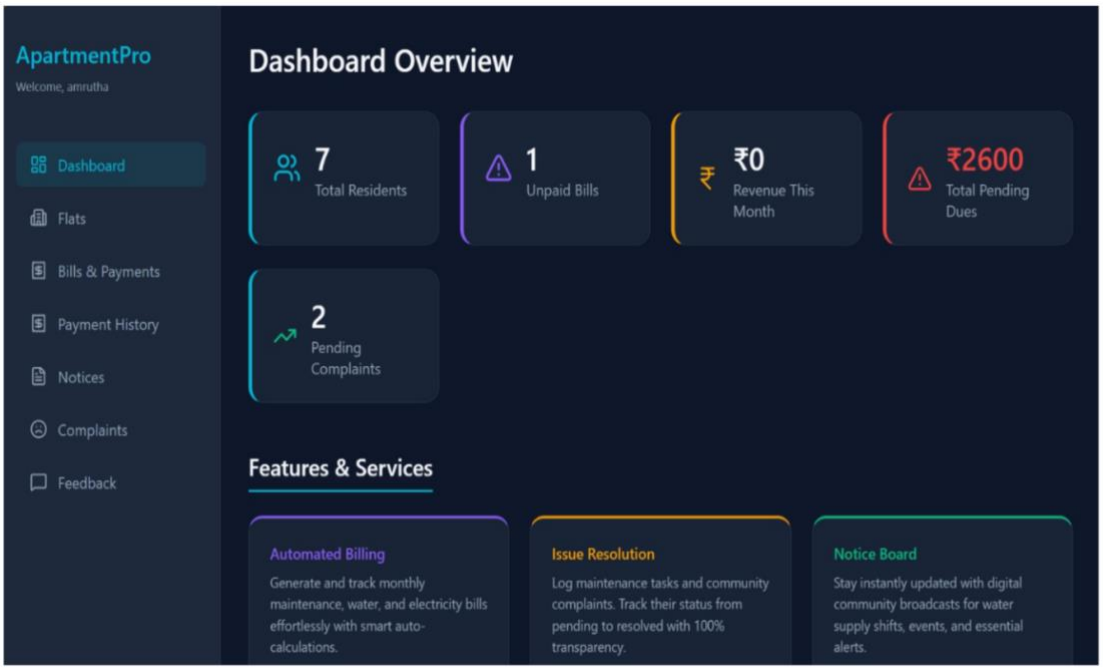
.env

README.md

- Contains project documentation
- Includes setup instructions and project overview

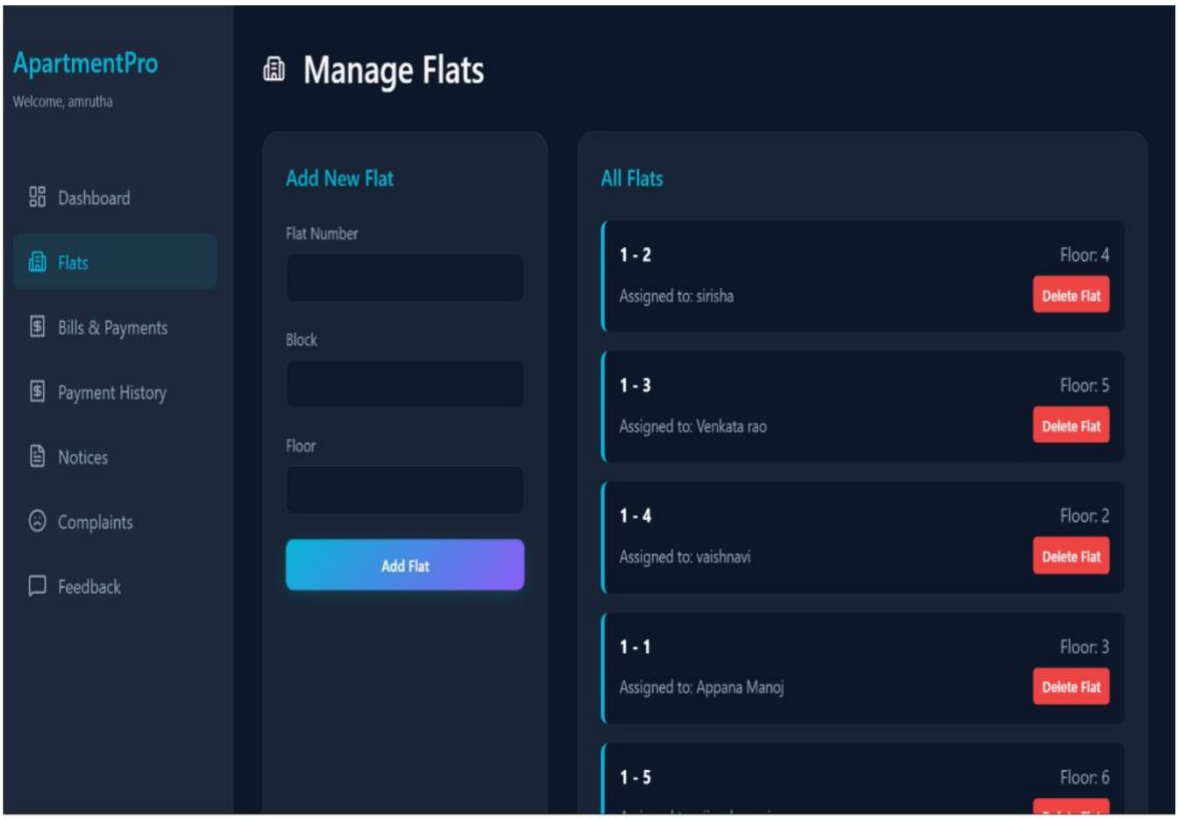
Output ScreenShots:

Admin Dashboard :



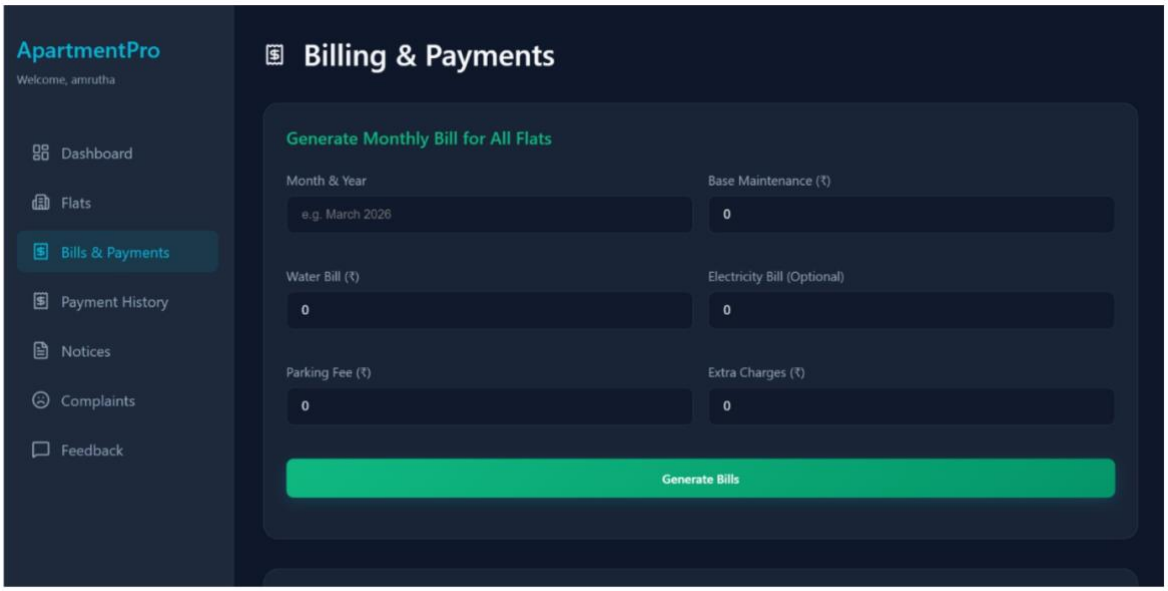
Admin dashboard

Admin related feature :

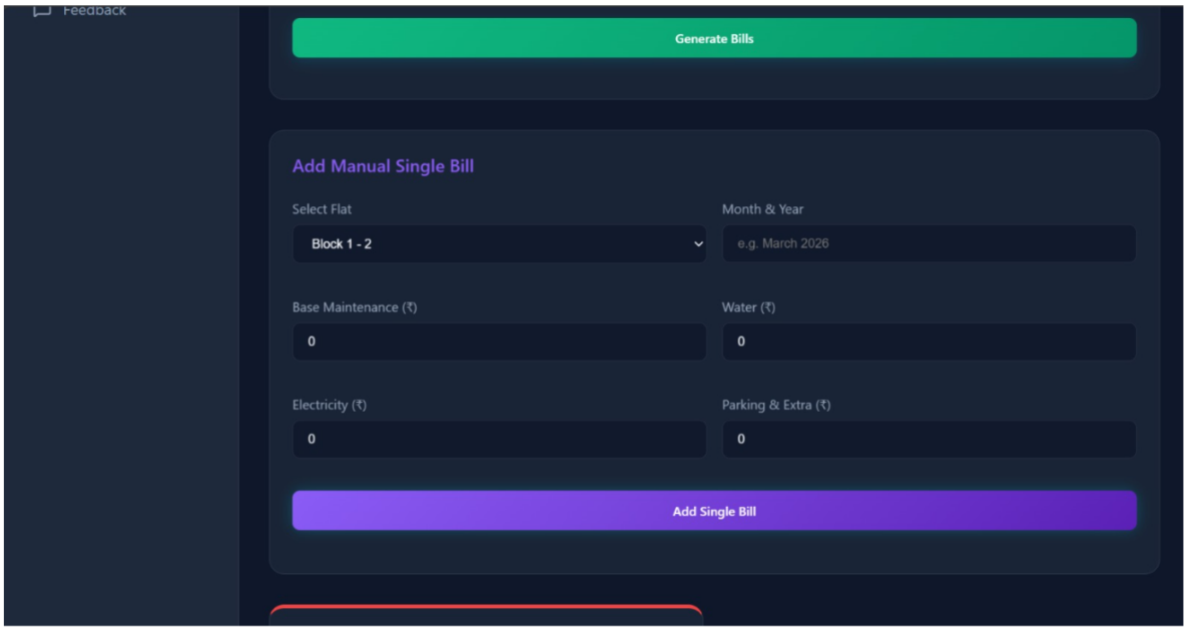


Admin related features

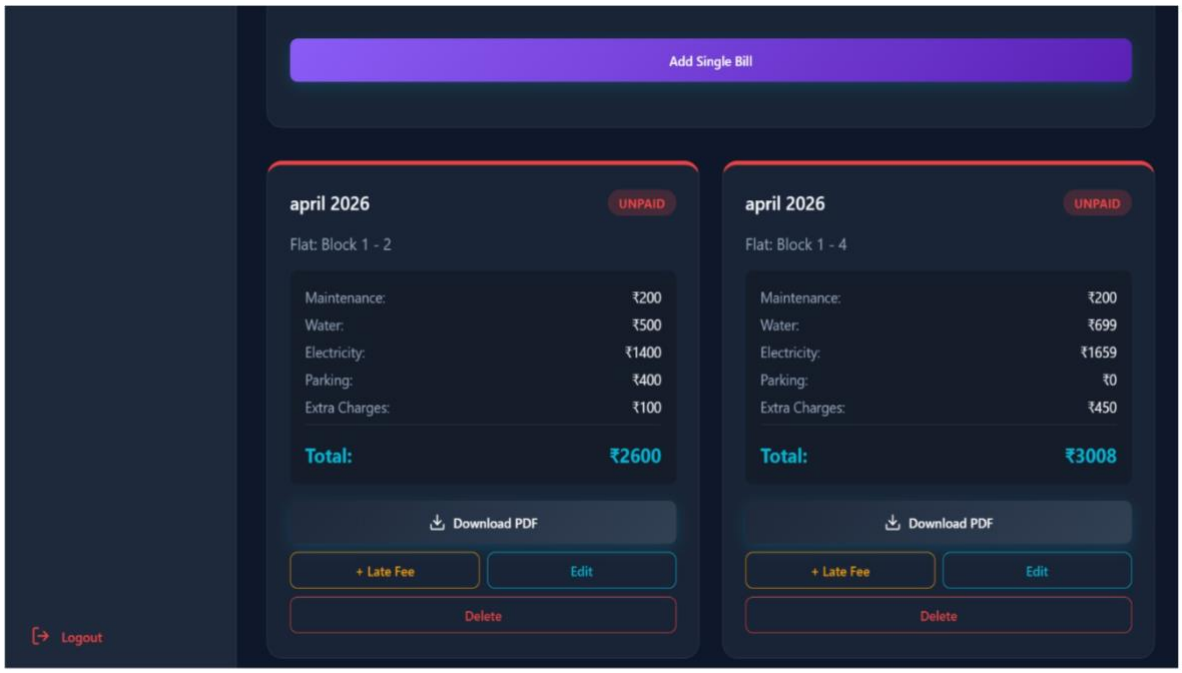
Billing System :



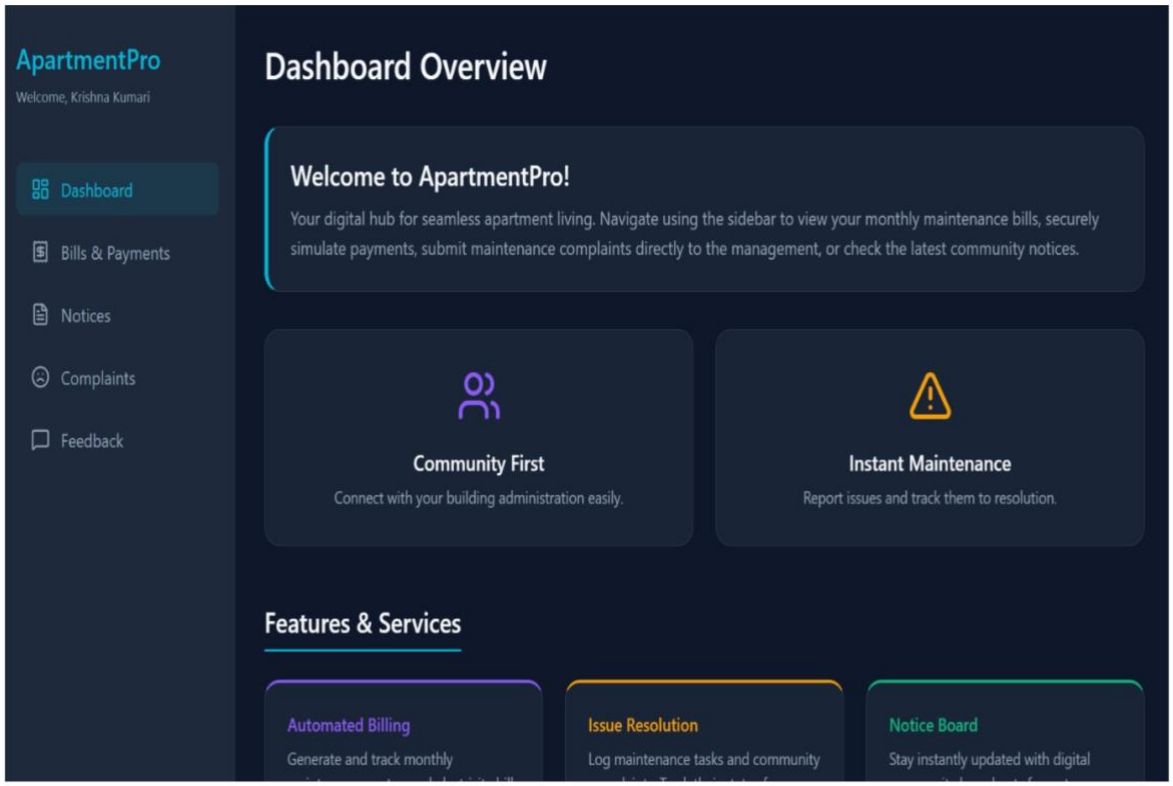
Billing System : Generate monthly bill for a single resident



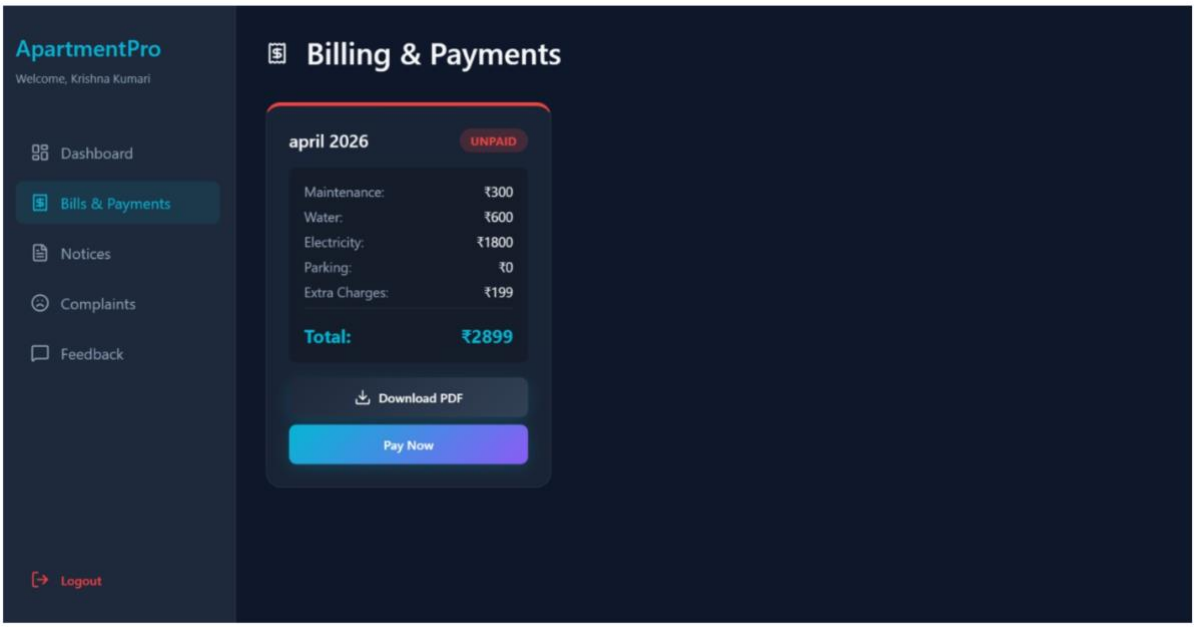
Billing status :



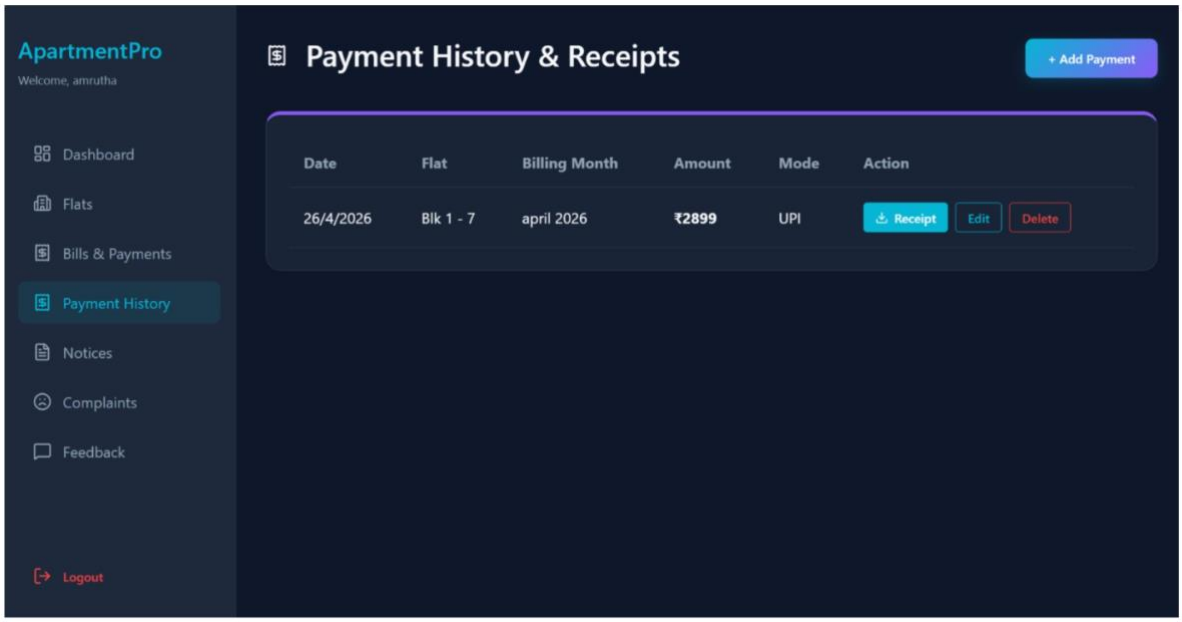
User(Resident) Interface :



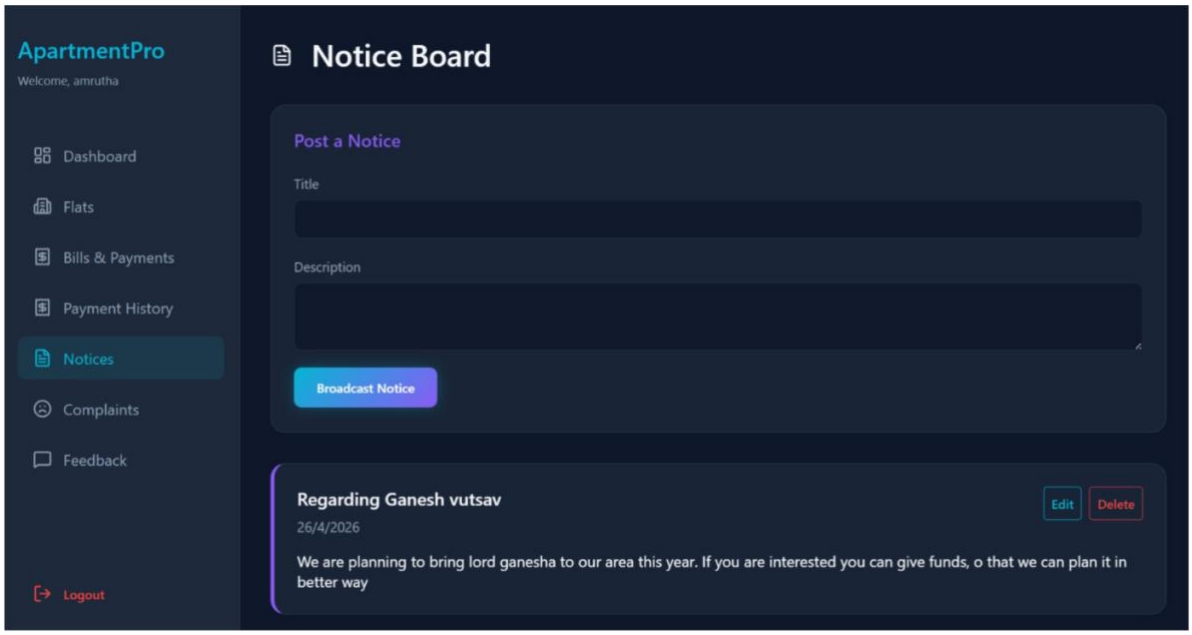
Billing in user interface :



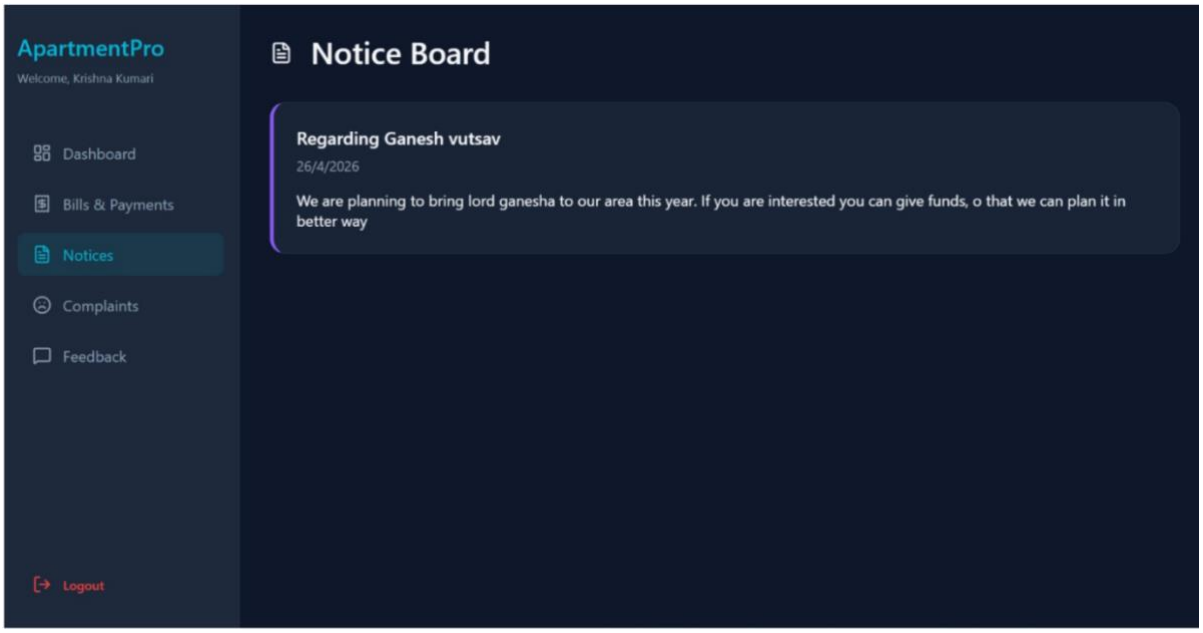
Payment History in Admin Dashboard :



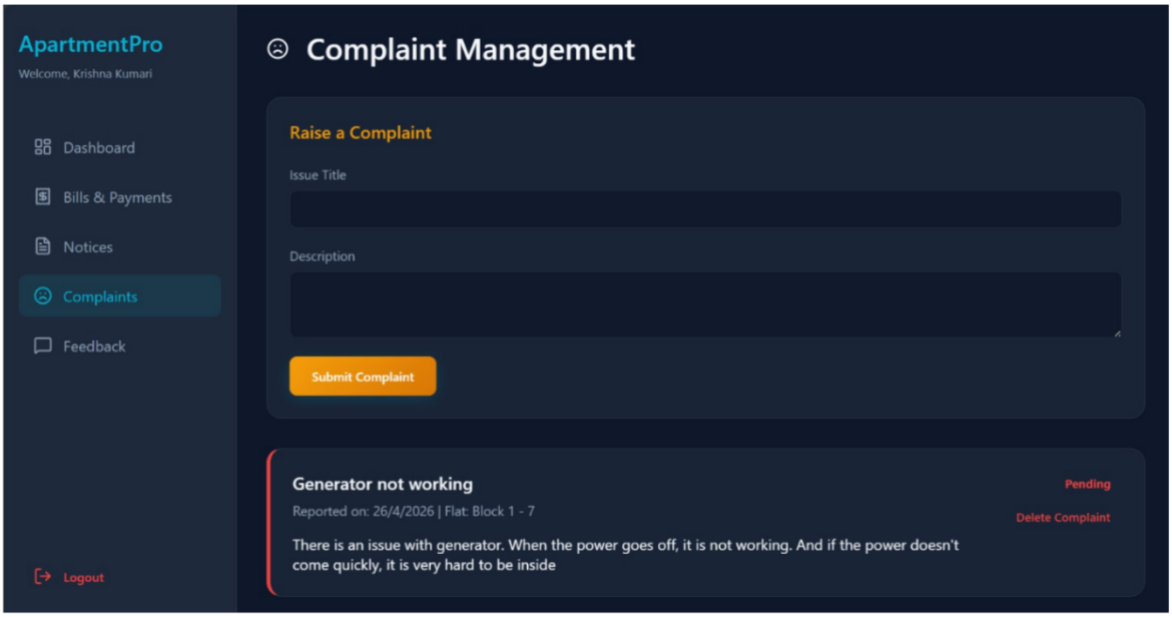
Notices page in Admin View :



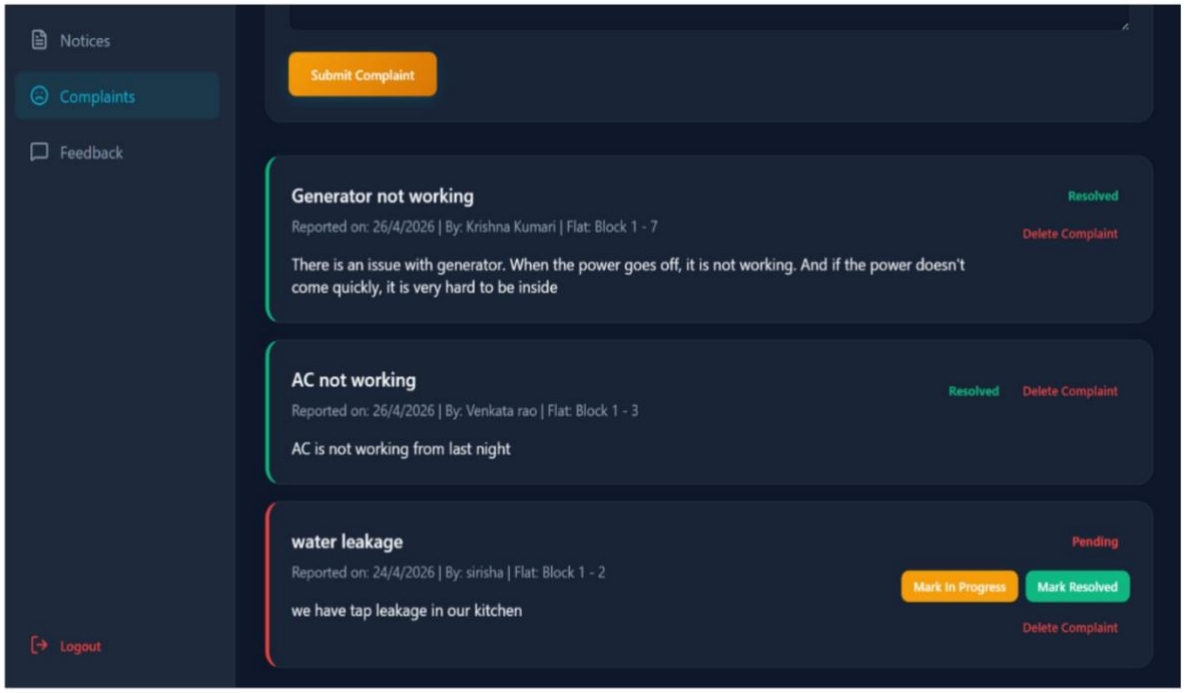
Notices in resident view :



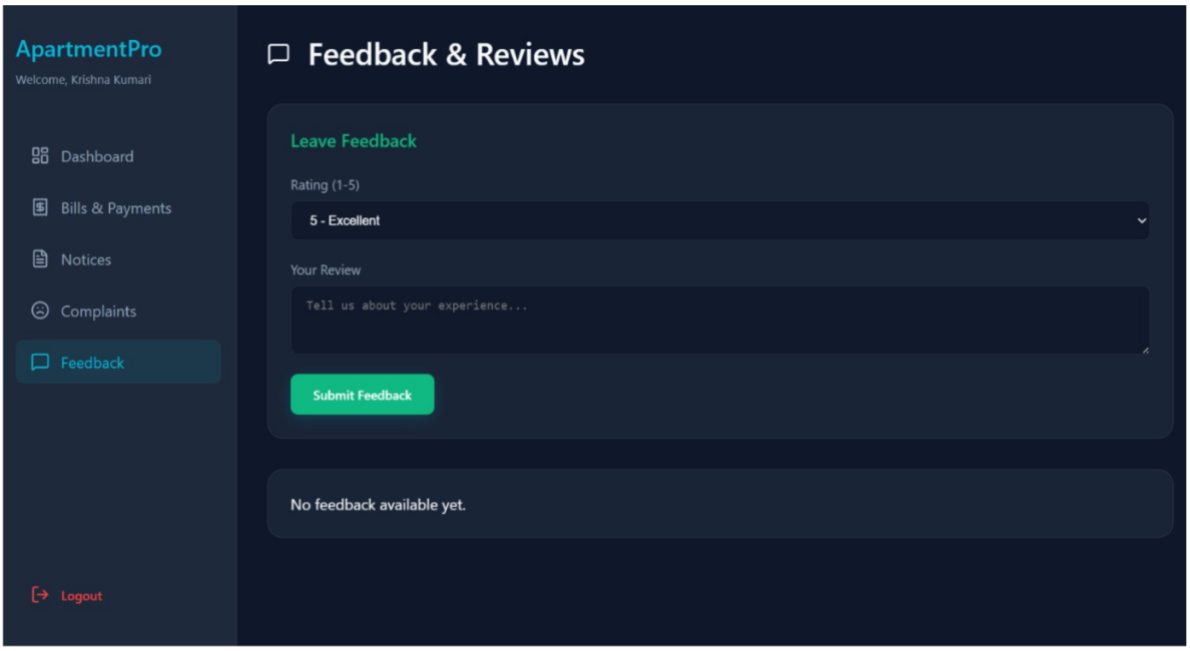
Complaint in user interface :



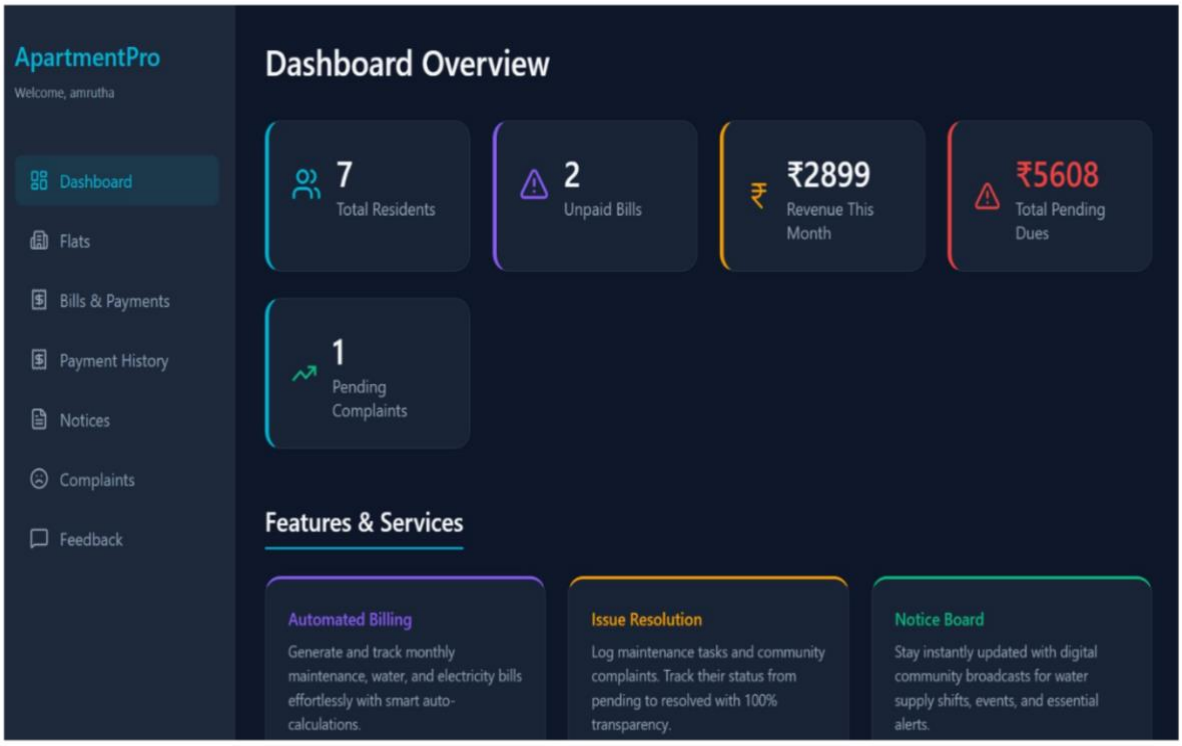
Complaints in Admin interface :

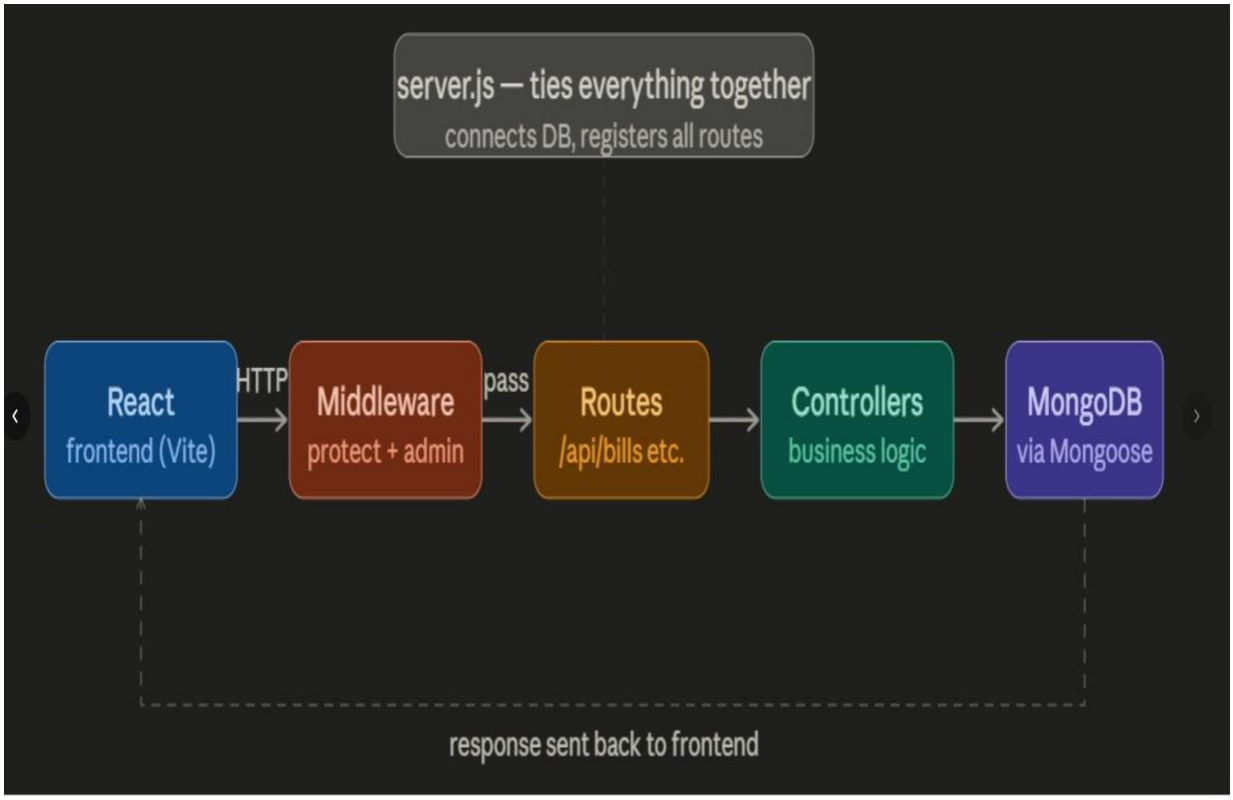


Feedback in user interface :



Admin dashboard after some addition of bills,payments and all :





Git hub link:

https://github.com/sahitya-koritala/Apartment-Maintenance-and-Billing-Portal-

Backend demo link:

https://github.com/sahitya-koritala/Apartment-Maintenance-and-Billing-Portal-/blob/main/Project Backend Explanation.md

Project overview link:

https://github.com/sahitya-koritala/Apartment-Maintenance-and-Billing-Portal-/blob/main/Project Overview Demo.md

Frontend link:

https://drive.google.com/file/d/1ezixNqDAS2ylnSz-Wpm7xzhcdWg3KuJ9/view?usp=drivesdk